TINY URL

url → www.abcdefghigk.com → www.tinyurl.com/abc

→ www.pqr.com/abcdefghi/tts → www.tinyurl.com/pqr

REQUIREMENTS

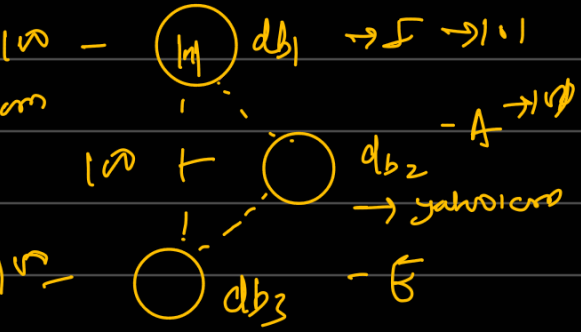
1) Unique
2) Count no. of clicks accurately.
3) clicks (10K – 100K)
4) 1 billion short urls
5) Read >> writes, want to optimize it
6) url length
8) Expiry time

SOLUTION

1) Using incremental number

www.tinyurl.com/1234
www.tinyurl.com/1235

problem
1) predictibility
2) Lack of uniqueness
3) limited <1 - 999999>



2) Using hashing function

$(0-9) + (a-z) + (A-Z) = 9 + 26 + 26 = 36 + 26 = 62$

Suppose we make url for length → 7

[(36) (36) (36) (36) (36) (36) (36)]

$\Downarrow 7$

$(36)^7 \quad \cdots\cdots \quad (36)^n$

if $n = 8$, we have $(36)^8$ = 2 trillion Combination

Hash Collision →

→ 10K data → for next data it will create a collision.

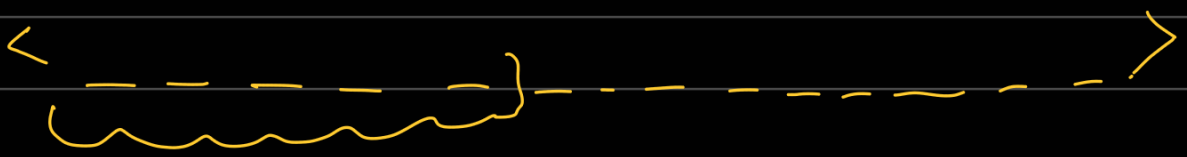
SHA-256 → possible combination $2^{(256)}$

↳ still collision rate is Extremely low
→ it is possible

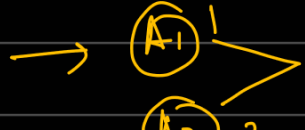
→ Randomization

1) B62 ——— 2) MD5

(collision will occur)



first 7 character



solution

1) maintain Counter

problem (A2)

1) it run out of number
2) single point failure

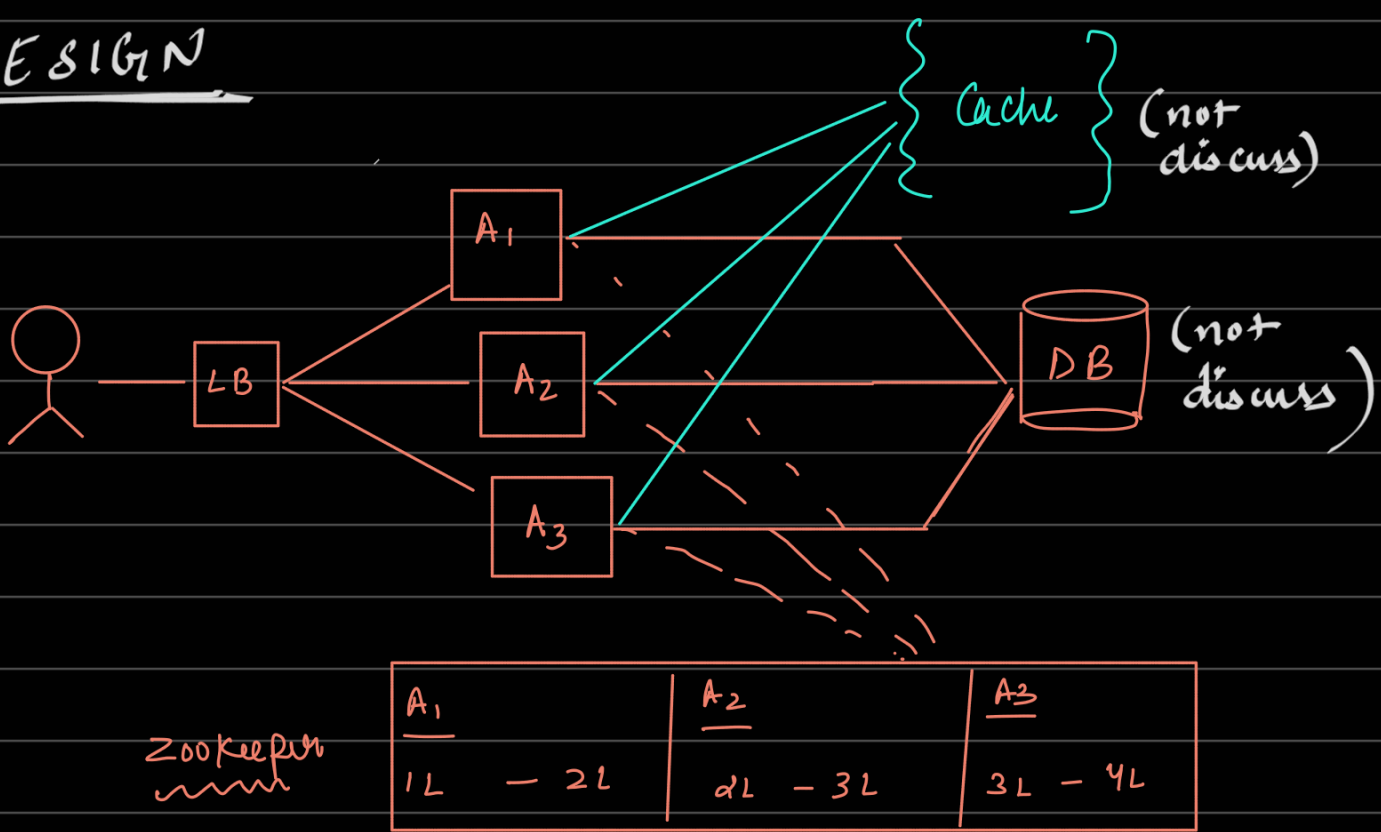
Best Solution

Bucket counter

1) will maintain Range
2) Apache Zookeeper

<1 — 1000> (A1)
<1000 — 2000> (A2)
<4000 - 8000> (A3)

DESIGN



Zookeeper

A1	A2	A3
1L — 2L	2L — 3L	3L — 4L

Database → (Single node)

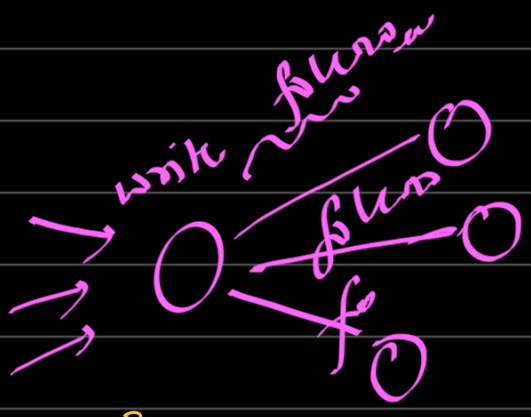
(Tenique)

tiny url	actual url	created at	Expiry at	Clicked
abc123	www.google.com	30/month/	2/April	500
Pqr456	www.hello.com			
Pqr456	www.hello.com			

→ indexed

↙

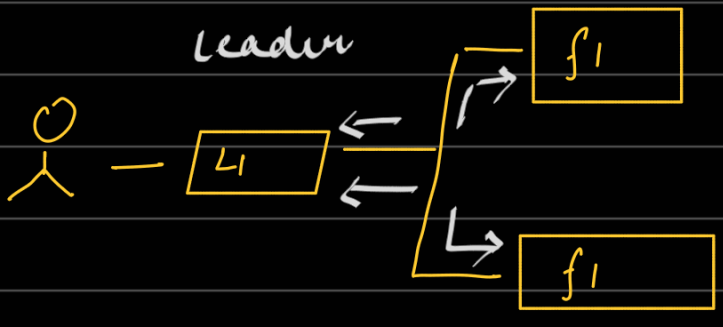
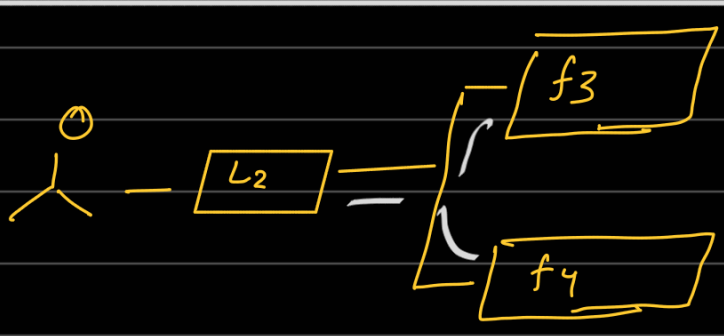
O(log(n)) vs (O(n))



Database Choice

1) Single leader Replication
2) partitioning (hash)
3) B-tree based

Relational	non-Relational
1) mySql (✓)	2) mongoDB (✗)

Synchronous	Asynchronous
	
1) write in all three then response will come	1) write in leader send Request to followers.
1) slow → (consistent)	2) fast - (inconsistent)

1) We need fast system
2) Go for asynchronous leader

Replication.

3) But Code - Logic to be Changed)

[Redis → follower DB → leader DB]

Caching →

1) Should we cache all the data?

→ Write Back → [1) write in cache
2) cache will write in DB]

problem
1) slow writes

→ write through → [1) put in DB
2) put in cache]

problem → Slow writes

→ write around →

(Asynchronous)

[1) put in DB
2) DB put in Redis
3) Got Result]

problem

1) which data to put in Redis.

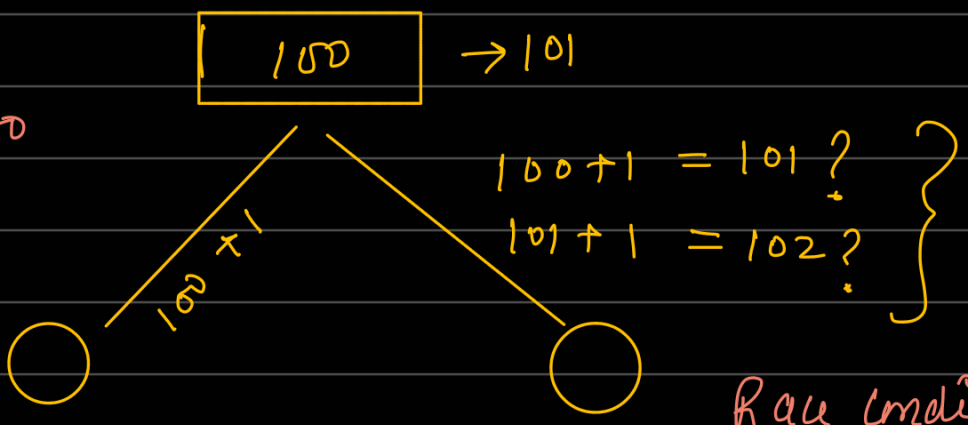
Solution

1) use → LRU

Clicking

1) Do we have to use locking?

2) Will it slow?



Race condition

Streaming Process

1) We can put data somewhere and aggregate them later

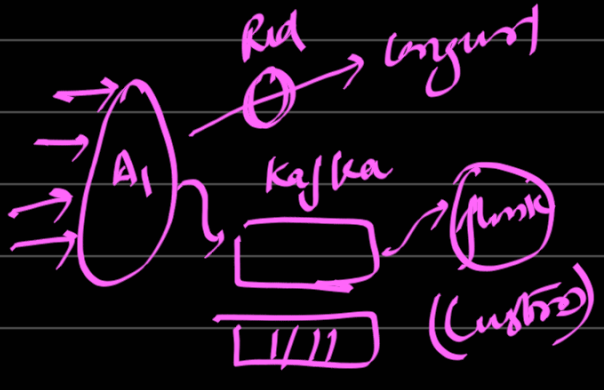
Solution

{ 1) time series DB
2) queue based → (Kafka) }

→ Batching → after every → 100
→ after some time →

→ (Spark Streaming)

(custom logic)



Deleting Expiry link

1) Cron job → if (currenttime > expiry time)
{
Delete
}

DB Redis

Complete Design